



AUDIT REPORT

July 2026

For



Table of Content

Executive Summary	05
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	11
Techniques and Methods	12
Types of Severity	14
Types of Issues	15
Severity Matrix	16
■ Critical Severity Issues	17
1. Court-order verification fails open outside a single hard-coded environment value	17
■ High Severity Issues	19
1. Recovery-request signature is bound to the caller's own destination address rather than to the account being recovered	19
2. Estate and privacy-code personal data is disclosed to any authenticated caller	21
3. Court-order verification reduces to a substring match and accepts a forged docket	23
4. Arbitrary attacker-supplied calldata is forwarded to on-chain execution under one opaque hash	25
5. Production administrative-only fulfillment guard is dead code and does not restrict fulfillment	27

6. Missing object-level authorization on the single-case read endpoint	29
■ Medium Severity Issues	31
1. Webhook registration accepts internal addresses and permits cross-tenant subscription	31
2. Token scope is not enforced, so a low-privilege token reaches every authenticated route	33
3. Account enumeration through non-uniform authentication and password-reset responses	35
4. No security response headers, no rate limiting, and large in-memory uploads	37
5. The cache tier is an untimed hard dependency on the entire case lifecycle	39
6. A single hot signer key held in an environment variable authorizes all on-chain recovery	41
7. Court-order documents are pinned to public IPFS	42
8. Server-side request forgery and untrusted document parsing in the court-records scraper	43
■ Low Severity Issues	44
1. Signed recovery and court-order messages carry no nonce, deadline, or case binding	44
2. Terms and will signatures are verified over a stringified payload and are replayable	46
3. Missing owner authorization on the webhook delivery-history endpoint	48
4. Privacy-code signature domain omits the chain identifier and fixes the verifying contract to the zero address	50
5. Unvalidated transaction value produces an uncaught error before signature verification	52
6. The change-password route has no authentication middleware and no session binding	54
7. Password change and reset do not revoke existing API keys or sessions	56

8. An estate-profile record is created for an arbitrary owner before the signature check	57
9. Credential-less admin access via hasBackend empty-string footgun (contingent)	59
10. New administrator passwords are returned in the HTTP response body	61
11. The administrative dashboard is exposed to cross-site request forgery	62
12. The wallet signing application ships without a content security policy or frame protection	63
13. Provider RPC keys can be compiled into the public client bundle	64
14. Dashboard authentication state is a forgeable client-side flag	65
15. The reviewer role is enforced only in the client and logout does not end the server session	67
16. The administrative single-page application ships without security response headers	68
17. The database schema is auto-altered on every service start	69
18. The dashboard has no server-side logout and diverges from the backend session	70
19. The administrative login screen advertises a credential-less access path	71
20. The will-upload signature step stringifies the file buffer before authorization	72
■ Informational Severity Issues	73
1. Recovered signer and owner addresses are written to debug logs	73
Recommendations to Complete Verification	74
Appendix A	75
Closing Summary & Disclaimer	76

Executive Summary

Client Jurat Systems

Applications recovery-backend, minimal-frontend, manual-review-dashboard

Repositories and Branches recovery-backend @ feature/automated-tests; minimal-frontend @ feature/automated-tests; manual-review-dashboard @ api-sdk

Live Environment Tested recovery-backend staging API (frontends not deployed for testing)

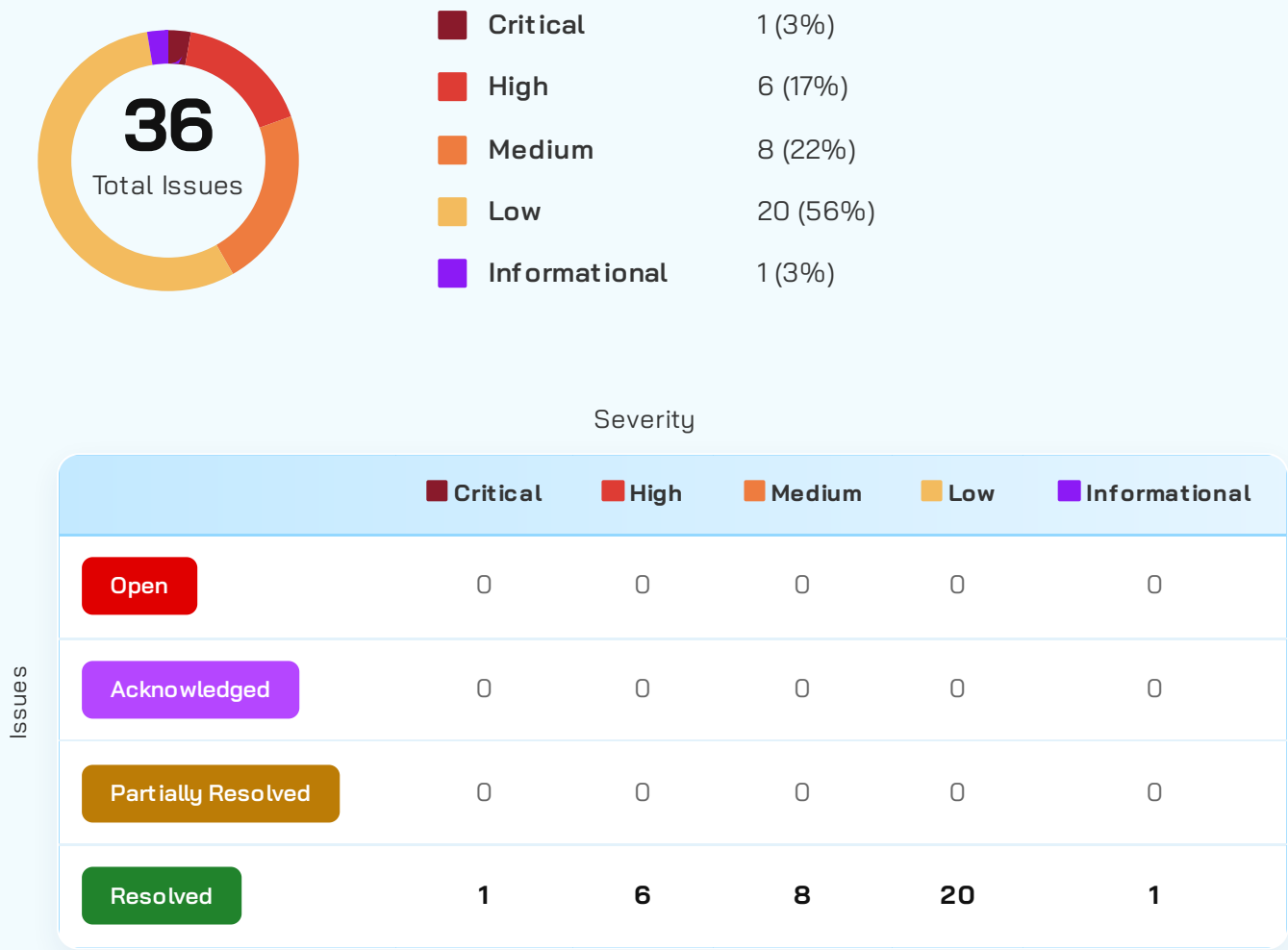
Assessment Period July 2026

Overview Jurat operates an asset-recovery protocol whose backend authorizes on-chain transactions that move assets out of a user's smart-contract wallet. A recovery request names a lost account and a recovery destination, the request is tied to a United States federal court order, and a human reviewer with administrative privilege then triggers on-chain execution. Because the platform moves value on behalf of third parties, this assessment concentrates on the controls that gate fund movement and on the confidentiality of the probate data the platform stores.

Most Significant Finding The most significant confirmed result is the recovery-request authorization model. The signature that creates a recovery case is verified against the caller's own recovery address rather than against the account being recovered. On staging we created a valid recovery case for a wallet we do not control, signed with our own key, and the server accepted it. Negative controls held, so the signature is genuinely verified. The defect is that the server requires no proof of control over the account whose assets would be moved. This is the entry condition for an asset-drain chain. The remaining links, a court-order check that fails open outside a single hard-coded environment value and an ineffective administrative authorization guard, are present in the code but could only be exercised with administrative credentials, which were not in scope for this phase.

Additional Findings Beyond the recovery chain, we confirmed cross-tenant disclosure of estate and probate data to any authenticated caller, acceptance of internal metadata addresses at webhook registration, absence of security response headers and rate limiting, and several weaknesses in the signing and session model. The client applications were reviewed in source only, because no frontend was deployed for testing.

Number of Issues per Severity



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Court-order verification fails open outside a single hard-coded environment value	Critical	Resolved
2	Recovery-request signature is bound to the caller's own destination address rather than to the account being recovered	High	Resolved
3	Estate and privacy-code personal data is disclosed to any authenticated caller	High	Resolved
4	Court-order verification reduces to a substring match and accepts a forged docket	High	Resolved
5	Arbitrary attacker-supplied calldata is forwarded to on-chain execution under one opaque hash	High	Resolved
6	Production administrative-only fulfillment guard is dead code and does not restrict fulfillment	High	Resolved
7	Missing object-level authorization on the single-case read endpoint	High	Resolved
8	Webhook registration accepts internal addresses and permits cross-tenant subscription	Medium	Resolved

Issue No.	Issue Title	Severity	Status
9	Token scope is not enforced, so a low-privilege token reaches every authenticated route	Medium	Resolved
10	Account enumeration through non-uniform authentication and password-reset responses	Medium	Resolved
11	No security response headers, no rate limiting, and large in-memory uploads	Medium	Resolved
12	The cache tier is an untimed hard dependency on the entire case lifecycle	Medium	Resolved
13	A single hot signer key held in an environment variable authorizes all on-chain recovery	Medium	Resolved
14	Court-order documents are pinned to public IPFS	Medium	Resolved
15	Server-side request forgery and untrusted document parsing in the court-records scraper	Medium	Resolved
16	Signed recovery and court-order messages carry no nonce, deadline, or case binding	Low	Resolved
17	Terms and will signatures are verified over a stringified payload and are replayable	Low	Resolved
18	Missing owner authorization on the webhook delivery-history endpoint	Low	Resolved
19	Privacy-code signature domain omits the chain identifier and fixes the verifying contract to the zero address	Low	Resolved
20	Unvalidated transaction value produces an uncaught error before signature verification	Low	Resolved

Issue No.	Issue Title	Severity	Status
21	The change-password route has no authentication middleware and no session binding	Low	Resolved
22	Password change and reset do not revoke existing API keys or sessions	Low	Resolved
23	An estate-profile record is created for an arbitrary owner before the signature check	Low	Resolved
24	Credential-less admin access via hasBackend empty-string footgun (contingent)	Low	Resolved
25	New administrator passwords are returned in the HTTP response body	Low	Resolved
26	The administrative dashboard is exposed to cross-site request forgery	Low	Resolved
27	The wallet signing application ships without a content security policy or frame protection	Low	Resolved
28	Provider RPC keys can be compiled into the public client bundle	Low	Resolved
29	Dashboard authentication state is a forgeable client-side flag	Low	Resolved
30	The reviewer role is enforced only in the client and logout does not end the server session	Low	Resolved
31	The administrative single-page application ships without security response headers	Low	Resolved
32	The database schema is auto-altered on every service start	Low	Resolved

Issue No.	Issue Title	Severity	Status
33	The dashboard has no server-side logout and diverges from the backend session	Low	Resolved
34	The administrative login screen advertises a credential-less access path	Low	Resolved
35	The will-upload signature step stringifies the file buffer before authorization	Low	Resolved
36	Recovered signer and owner addresses are written to debug logs	Informational	Resolved

Checked Vulnerabilities

✓ Authorization & Access Control

✓ Object-Level Authorization (IDOR)

✓ EIP-712 Signature Verification & Replay Protection

✓ Session, Token & API-Key Management

✓ Server-Side Request Forgery (SSRF)

✓ Input Validation & Error Handling

✓ Rate Limiting & Brute-Force Protection

✓ Security Response Headers

✓ Data Protection & PII Confidentiality

✓ Private Key & Secrets Management

✓ Cross-Site Request Forgery (CSRF)

✓ Client-Side Storage & Local Auth State

✓ Content Security Policy & Clickjacking Protection

✓ Third-Party & Infrastructure
Dependency Resilience (Cache, IPFS, RPC)

✓ Business-Logic / Recovery-Chain Integrity

✓ Database Migration & Schema Management

✓ Credential Exposure in Responses & Logs

Techniques and Methods

Throughout the assessment, care was taken to ensure:

- Testing remained non-destructive and confined to the authorized staging scope.
- No funds were moved, no data was destroyed, and no denial-of-service load was generated.
- Verification used throwaway accounts and inert test records only; webhooks were registered but never triggered.
- No third-party infrastructure - the court records service, the pinning service, the RPC providers, or the on-chain contract - was targeted.

The following techniques, methods, and tools were used to review the applications:

Static Source-Code Review

The static phase reviewed the source of all three repositories for authorization, cryptographic verification, input handling, and data protection defects.

Dynamic Verification Against Staging

The dynamic phase attempted to confirm, refute, or re-scope each finding against the running staging backend, using throwaway accounts and non-destructive requests.

Adversarial Review

Each candidate finding was subjected to an adversarial review pass to separate genuine vulnerabilities from intended behavior and false positives.

Tools and Platforms Used for Audit

Manual source review; curl-based verification against the staging API; ethers.js v6 for EIP-712 typed-data signature generation (see Appendix A).

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical

Direct and practical path to unauthorized movement of user assets, or equivalent loss of control of the protocol.

■ High

A defect that materially weakens a fund-movement or data-protection control, or a precondition that composes into asset loss under conditions present in the system.

■ Medium

A defect that exposes sensitive data, enables server-side request forgery, or removes a defense-in-depth control without a single-step path to loss.

■ Low

A localized weakness or hardening gap with limited standalone impact.

■ Informational

An observation or hygiene item with no direct security impact.

Types of Issues

Confirmed on Staging

Reproduced against the live backend with captured request and response evidence.

Confirmed in Source

Established by code review; not exercisable in this phase because it requires a deployed frontend, privileged credentials, or the restored cache.

Intended or Accepted

Real behavior that reflects a design decision or an accepted hardening gap with bounded residual risk.

Not a Vulnerability

A candidate that carries no security impact on examination.

Severity Matrix

		Impact		
		 High	 Medium	 Low
Likelihood	 High	Critical	High	Medium
	 Medium	High	Medium	Low
	 Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Critical Severity Issues

Court-order verification fails open outside a single hard-coded environment value

Resolved

Confirmed in source; live proof requires privileged credentials

Path

```
recovery-backend/src/services/court.services.ts:707-716
```

Description

The function that decides whether a fetched court document authorizes a recovery returns true whenever the environment value is not one of two hard-coded strings. The intended production value in the codebase is upper case, while the check compares against the lower case literal. On any staging or development deployment, and plausibly in production because of that casing mismatch, the check is skipped and every pending case verifies with no genuine court order. This is the authorization anchor for fund movement.

```
verifyContentHashValidation(documentContent: string, jrcHash: string): boolean {
  let hash: string = jrcHash;

  if (process.env.JURAT_ENV === JURAT_ENV.localTesting || process.env.JURAT_ENV === 'production') {
    hash = `${jrcHash}-1`;
    return documentContent.includes(hash);
  } else {
    return true;
  }
}
```

Court-order verification fails open outside a single hard-coded environment value *(continued)*

Steps to Reproduce

Reproduction of the fail-open branch requires an administrative account, because it is reached only from the fulfillment path. As a standard user the fulfillment routes return forbidden. The unauthenticated environment oracle indicates the deployed value is not the local-testing string.

Proof of Concept

```
curl -s -b jar.txt -X PATCH $BASE/recovery-cases/review/fulfill/<caseId>  
Observed as a standard user: 403 {"error":{"code":"UNAUTHORIZED"}}
```

Environment oracle (unauthenticated):

```
curl -s -X PATCH $BASE/auth/forgot-password -H 'Content-Type: application/json' -d '{"email":"<known>"}
```

Observed: 201 {"message":"Recovery link sent","token":null}. A null token indicates the deployed environment value is not the local-testing string, so the court check depends on the lower-case literal.

Verification

The fund-movement path is restricted to administrative roles, so the fail-open branch could not be executed as a standard user. The environment oracle described in the account-enumeration finding confirms the deployed value is not the local-testing string, which is consistent with the fail-open branch being reachable on staging. Full confirmation requires an administrative account.

Recommendation

Make the check fail closed with a default of false, and compare the environment value against the shared constant after normalizing case at startup. Reject unknown environment values at boot. Never allow a mode in which verification always passes on a deployed system.

High Severity Issues

Recovery-request signature is bound to the caller's own destination address rather than to the account being recovered

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/services/ethers.services.ts:99-127`

Description

The signature that creates a recovery case is verified against the recovery destination address chosen by the caller, not against the lost account whose assets would be moved. The design intent is that a user who has lost a key cannot sign with it, so the request is signed with the new key. The consequence is that the server accepts a recovery request for any account, with no proof of control over that account. The only intended compensating control is the downstream court order, which itself fails open.

```
const verifiedMsg = ethersServices.verifySignedCreationInfoMessage(  
    caseInfo.recoveryAccount,  
    caseInfo,  
    transactionsInfo,  
    signature  
);  
  
if (!verifiedMsg) throw new InvalidSignerError();
```

Recovery-request signature is bound to the caller's own destination address rather than to the account being recovered *(continued)*

Steps to Reproduce

1. Obtain a standard session (setup above). 2. Produce a signature with the helper in Appendix A, naming a lost account you do not control and your own generated key as the recovery account. 3. Submit the case. 4. As controls, resubmit with one hex nibble of the signature changed, and again with the lost account swapped.

Proof of Concept

```
curl -s -b jar.txt -X POST $BASE/recovery-cases/new-case -H 'Content-Type: application/json' \
-d '{"case":{"lostAccount":"0xFA77..5Ff4","recoveryAccount":"0xBE8C..86Ba6","chainId":84532,
    "reasonForRecovery":"lost key","email":null},
    "transactions":[{"dest":"0x00..dEaD","value":"0","data":"0x"}],
    "signature":"0x<from Appendix A>"}
```

Observed: 200 {"caseId":"856bccd4-6a97-4812-a72f-f4838fc8f730","jrcHash":"0x1f5d53f3..."}

Controls: tampered signature returns 401 INVALID_SIGNER; lost account swapped returns 401 INVALID_SIGNER.

Verification

Confirmed on staging. A recovery case was created for a lost account not under our control, signed with our own recovery key, and the server returned success with a case identifier and hash. Negative controls held: a one-nibble change to the signature and a swap of the lost account both returned an invalid-signer error, which proves the signature is genuinely verified and isolates the defect to the missing binding.

Recommendation

Do not treat the creation signature as authorization to move funds. Require an independent and verifiable link between the on-chain lost account and a parsed, genuine court order that names that account and destination. Where the recovery reason is not a lost key, require a co-signature or an on-chain pre-authorization from the lost account.

High Severity Issues

Estate and privacy-code personal data is disclosed to any authenticated caller

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/controllers/estate.controller.ts:103-119,203-230`

Description

The estate-profile read and the privacy-code resolution endpoint are authenticated but carry no binding between the caller and the data. Any authenticated caller can retrieve an owner's agreement email, agreement signature, terms-acceptance timestamps, will-document metadata, and the linkage between an owner address and a Jurat wallet.

```
async getProfile(req: Request, res: Response, next: NextFunction) {
  try {
    const ownerAddress = req.params.ownerAddress as string;
    const profile = await JuratEstateProfileModel.findOne({
      where: { ownerAddress: ownerAddress },
      logging: false,
    });

    if (!profile) {
      return res.status(200).json({ profile: null });
    }

    res.status(200).json({ profile: await serializeEstateProfile(profile) });
  } catch (err) {
    next(err);
  }
}
```

Estate and privacy-code personal data is disclosed to any authenticated caller *(continued)*

Steps to Reproduce

1. As user A, create an estate profile for an owner address (accept the agreement and add a privacy code). 2. Register and log in an unrelated user B. 3. As B, read the same owner profile.

Proof of Concept

```
# as user B, reading user A's owner address
curl -s -b jarB.txt $BASE/estate-profiles/0x1197cB45...Bd1b2
Observed: 200 with agreementEmail "victim.estate.owner@example.com", termsAcceptedAt,
agreementSignature, and privacyCodes[.{"name:"Alice Victim Testcase", state:"New York",
juratWalletAddress:"0x62d0...4832"}]. No ownership check.
```

Verification

Confirmed on staging. A profile was populated as one user and then read in full by an unrelated second user, returning the owner email, the privacy-code name and state, and the wallet linkage. The data is stored in the primary database and was unaffected by the cache outage. This directly contradicts the platform's own terms, which state that a privacy-code address is disclosed only under a court order.

Recommendation

Require proof of control of the owner address, or an owning session, to read a profile. Return the minimum necessary data for claimant discovery and authorize the resolution flow.

High Severity Issues

Court-order verification reduces to a substring match and accepts a forged docket

Resolved

Confirmed in source; live proof requires privileged credentials

Path

```
recovery-backend/src/services/court.services.ts:288,707-712
```

Description

Even when the environment gate is satisfied, the entire court-order control reduces to checking whether the extracted document text contains the case hash followed by a fixed suffix. There is no verification of the filer, the authenticity of the order, whether the order was granted, or that the case pertains to the lost account. The finality analysis that would perform some of these checks is present but commented out.

```
// Check if the jrcHash is present in the document content
    // If ${jrcHash}-1 is present, return true. If ${jrcHash}-0 or not
    // present, return false.
    return this.verifyContentHashValidation(docketDocContent, jrcHash);
```

Court-order verification reduces to a substring match and accepts a forged docket *(continued)*

Steps to Reproduce

Reached only from the administrative fulfillment path, so confirmed in source. The case hash that the docket text must contain is derivable before filing through the open hash endpoint.

Proof of Concept

```
curl -s -b jar.txt -X POST $BASE/recovery-cases/jrc-hash -H 'Content-Type: application/json' \
  -d '{"rec":{"..."},"txs":[{"...}]}'
```

Observed: 200 {"jrcHash":"0x..."}.

The document need only contain this value followed by the fixed suffix.

Verification

The check is reached only through the administrative fulfillment path, which returns forbidden to a standard user, so it was confirmed in source rather than executed. The case hash is derivable by the requester before filing through an open endpoint, which makes the required string predictable.

Recommendation

Bind verification to a specific court, case, and docket, to an authenticated and final granting order, and to the specific lost account and destination. Re-enable and harden the finality analysis and treat parsed document text as untrusted.

High Severity Issues

Arbitrary attacker-supplied calldata is forwarded to on-chain execution under one opaque hash

Resolved

Confirmed in source; live proof requires privileged credentials

Path

```
recovery-backend/src/services/ethers.services.ts:21-75
```

Description

The stored transaction destination, value, and calldata are forwarded verbatim into the on-chain execution call with no destination allow-list, no value cap, and no restriction on calldata. The court authorization is a match on an opaque hash, so the human reviewer approves a digest while the system executes whatever transactions hash to it.

```
const txs = [];  
  for (let i = 0; i < txInfo.length; i++) {  
    txs.push({ dest: txInfo[i].dest, value: txInfo[i].value, data: txInfo[i].data });  
  }
```

Arbitrary attacker-supplied calldata is forwarded to on-chain execution under one opaque hash *(continued)*

Steps to Reproduce

1. Obtain a session and a valid signature (Appendix A) but set the transaction data to an attacker-authored call.
2. Submit the case and observe that the calldata is accepted and stored.

Proof of Concept

```
transactions:[{"dest":"0x00..dEaD","value":"0",  
    "data":"0xa9059cbb00000000000000000000000000000000000000000000000000000000  
000000001e240"}]  
Observed: 200 {"caseId":"374a6bd6...","jrcHash":"0x59157a37..."}. A 138-char  
acter token-transfer  
calldata is accepted at creation. Execution is administrative-gated and was  
not run.
```

Verification

The input half is confirmed on staging: a 138-character attacker-authored token-transfer calldata was accepted at case creation and returned success. Execution is administrative-gated and was not run. The realized magnitude also depends on the out-of-scope on-chain contract.

Recommendation

Bind the court order to decoded, human-readable transaction intent and re-derive it at fulfillment.
Validate transactions before submission and render the decoded calls in the reviewer interface.

High Severity Issues

Production administrative-only fulfillment guard is dead code and does not restrict fulfillment

Resolved

Confirmed in source; live proof requires privileged credentials

Path

`recovery-backend/src/controllers/case.controller.ts:555-567`

Description

The guard that is meant to restrict production fulfillment to the administrator role is non-functional for three reasons. It reads a request property that is never populated, it does not return after sending the unauthorized response so execution continues, and it compares the environment value against the wrong casing. The base role check that limits access to administrator and master roles does still function.

```
const role = localsAuth.role;
    if (!(role === UserRoles.master || role === UserRoles.admin)) throw
new UnauthorizedError();

    const caseId = req.params.id as string;
    const cacheKey = `case_${caseId}`;

    // Note: we allow master to fulfill request in dev mode
    if(process.env.JURAT_ENV === 'production' && req.role !== UserRoles.
admin) {
        res.status(401).send("Not an admin");
    }
```

Production administrative-only fulfillment guard is dead code and does not restrict fulfillment *(continued)*

Steps to Reproduce

As a standard user, call each fulfillment and review route and observe the forbidden response, which confirms the base role check. The dead guard that is meant to separate the administrator and master roles could not be exercised without an administrative account.

Proof of Concept

```
curl -s -b jar.txt -X PATCH $BASE/recovery-cases/review/fulfill/<id> # 403  
UNAUTHORIZED  
curl -s -b jar.txt -X PATCH $BASE/recovery-cases/review/reject/<id> # 403  
UNAUTHORIZED
```

Verification

Confirmed that a standard user receives forbidden on every fulfillment and review route. The master-versus-administrator separation that this guard is meant to enforce could not be exercised without an administrative account.

Recommendation

Return after every authorization failure, read the authenticated role from the response locals rather than the request, and compare against the shared environment constant.

High Severity Issues

Missing object-level authorization on the single-case read endpoint

Resolved

Confirmed in source; live proof blocked by staging cache outage

Path

```
recovery-backend/src/controllers/case.controller.ts:97-126
```

Description

The single-case read endpoint is authenticated but performs no ownership or role check, so any authenticated principal, including the anonymous interface token, can read any case, including the lost and recovery addresses, the email, the recovery transactions, and the court information. A parameter-name mismatch also causes the lookup to ignore the identifier in the path.

```
async getCase(req: Request, res: Response, next: NextFunction) {  
  const caseId = req.params.id;  
  const cacheKey = `case_${caseId}`;  
  
  try {
```

Steps to Reproduce

1. Obtain a session. 2. Read any case by identifier. On staging the cache was unavailable, so the endpoint returned a gateway timeout. The sibling estate read, which does not use the cache, confirmed the same missing-ownership pattern live (see finding 3).

Proof of Concept

```
curl -s -b jar.txt $BASE/recovery-cases/<any-caseId>  
Observed: 504 Gateway Timeout (cache dependency, see finding on the cache ti  
er).
```

Missing object-level authorization on the single-case read endpoint*(continued)***Verification**

The endpoint reads from the cache first and the staging cache was unavailable, so every request returned a gateway timeout. The missing authorization is unchanged in source, and the sibling estate read, which does not use the cache, confirmed the same missing-ownership pattern live. Confirmation of this specific endpoint requires the cache to be restored.

Recommendation

Enforce object-level authorization so that a case is readable only by an administrator or by the principal bound to it, and correct the path-parameter name.

Medium Severity Issues

Webhook registration accepts internal addresses and permits cross-tenant subscription

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/controllers/webhook.controller.ts:10-33`

Description

Webhook registration validates the target only as a syntactic URL, with no block on loopback, private, link-local, or cloud-metadata ranges, and it does not verify that the caller owns the case identifier it is registered against. The signing secret is returned to the registrant. The realized impact today is bounded because a separate defect in the delivery worker appears to prevent the outbound request from being sent, but that is a one-line reliability fix away from an active server-side request forgery with no compensating control.

```
async createWebhook(req: Request, res: Response, next: NextFunction) {
    const inputs = WebhookCreationInputSchema.safeParse(req.body);
    if(!inputs.success) throw new InputValidationError();

    const { url, caseId, label } = inputs.data;

    const auth = res.locals.auth as AuthPayload;

    try {
        const events = [...Object.values(WebhookEvents)]

        const output = await webhookServices.registerWebhook(
            caseId,
            auth.sub,
            url,
            events,
            label
        )

        res.status(201).json({ webhookId: output.id, webhookSecret: output.secret });
    } catch (err) {
        next(err);
    }
}
```

Webhook registration accepts internal addresses and permits cross-tenant subscription *(continued)*

Steps to Reproduce

1. As user A, register a webhook whose target is the cloud-metadata address, against A's own case. 2. As an unrelated user B, register a webhook against A's case identifier. Deliveries are not triggered.

Proof of Concept

```
curl -s -b jar.txt -X POST $BASE/webhooks -H 'Content-Type: application/json' \
  -d '{"url":"http://169.254.169.254/latest/meta-data/","caseId":"<A-case>","label":"poc"}'
Observed: 201 {"webhookId":"0bb60e94...","webhookSecret":"whsec_f17e5042..."}.
```

Cross-tenant: user B against A's caseId returns 201 as well. A non-existent caseId returns 500.

Verification

Confirmed on staging. A webhook targeting the cloud-metadata address was accepted and returned a webhook identifier and signing secret. A second tenant registered a webhook against the first tenant's case identifier and was also accepted, which confirms both the internal-address acceptance and the cross-tenant subscription. A non-existent case identifier returned a database error, so the case must merely exist rather than be owned.

Recommendation

Verify case ownership before registration, and enforce an egress allow-list that blocks loopback, private, link-local, and metadata ranges with DNS re-validation on connect and a requirement for public HTTPS targets. Correct the delivery-worker association before that control becomes the only barrier.

Medium Severity Issues

Token scope is not enforced, so a low-privilege token reaches every authenticated route

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/middleware/auth.ts:77-84`

Description

The anonymous interface token is minted with a create-only scope and the API-key path assigns a fixed scope, but no route or middleware ever checks scope, so the least-privileged token reaches every authenticated endpoint. All anonymous sessions also share one identity, which collapses per-user isolation for any record keyed on that identity.

```
return {
  sub:      keyRecord.userId,
  type:     'user',
  scope:    ['read', 'write'],
  //email:   record.user.email,
  authMethod: 'api_key',
  role:     'user'
};
```

Steps to Reproduce

1. Obtain a standard user session, whose token scope is create-only. 2. Call routes unrelated to that scope and observe that they are reached.

Proof of Concept

```
curl -s -b jar.txt $BASE/estate-profiles/0x<any>      # reached, 200
curl -s -b jar.txt -X POST $BASE/webhooks -d '{...}'  # reached
No route enforces the token scope.
```

Token scope is not enforced, so a low-privilege token reaches every authenticated route *(continued)***Verification**

Confirmed on staging. A standard user token reached the case-creation, estate-read, and webhook-registration routes, none of which relate to its scope.

Recommendation

Enforce scope and token type per route through a middleware factory, deny anonymous tokens on read and administrative routes, and never key per-user records on a shared identity.

Medium Severity Issues

Account enumeration through non-uniform authentication and password-reset responses

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/controllers/auth.controller.ts:68,71,276`

Description

The authentication and password-reset flows return distinct responses for known and unknown accounts. An unknown login returns not-found while a wrong password returns unauthorized, and the reset flow returns a different status for known and unknown emails. No rate limiting is present.

```
const user = await JuratUserModel.findOne({
  where: { email }, logging: false
});

if (!user) throw new UserNotFoundError();

const isMatch = await bcrypt.compare( password, user.password );
if (!isMatch) throw new InvalidPasswordError();
```

Account enumeration through non-uniform authentication and password-reset responses *(continued)*

Steps to Reproduce

Send authentication and reset requests for a known and an unknown account and compare the responses. Send repeated failed logins and observe the absence of throttling.

Proof of Concept

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST $BASE/auth/login \
  -d '{"email":"nobody@example.com","password":"Passw0rd!x"}'      # 404 USER_
NOT_FOUND
curl -s -o /dev/null -w '%{http_code}\n' -X POST $BASE/auth/login \
  -d '{"email":"tester@example.com","password":"wrong"}'          # 401 INVA
LID_PASSWORD
forgot-password: unknown returns 404, known returns 201. Twelve rapid failed
logins returned no 429.
```

Verification

Confirmed on staging across four endpoints, with the reset endpoint reachable without authentication. Twelve rapid failed logins produced no lockout or throttling response.

Recommendation

Return uniform responses for authentication and reset flows regardless of account existence, and add rate limiting and backoff, starting with the dependency that is already present but unused.

Medium Severity Issues

No security response headers, no rate limiting, and large in-memory uploads

Resolved

Confirmed on staging (dynamic test)

Path

recovery-backend/src/app.ts:30-46

Description

The service sends no content security policy, no frame options, no strict transport security, and no content-type options. No rate limiting is wired despite the dependency being present, and will uploads are buffered fully in memory with a fifty-megabyte limit and an accept-all filter.

```
private config(): void {
  const corsOrigins = [
    process.env.USER_INTERFACE_URL,
    process.env.MANUAL_REVIEWER_URL,
    ...(process.env.INTEGRATOR_ORIGINS ?? "").split(","),
  ].filter((origin): origin is string => Boolean(origin && origin.trim(
  )))
    .map(origin => origin.trim());

  const corsOptions = {
    origin: corsOrigins,
    credentials: true
  }

  this.app.use(cors(corsOptions));
  this.app.use(cookieParser());
  this.app.use(express.json());
  this.app.set('trust proxy', 1);
}
```

No security response headers, no rate limiting, and large in-memory uploads *(continued)*

Steps to Reproduce

Request any endpoint and inspect the response headers. Send repeated failed logins and observe the absence of a throttling response.

Proof of Concept

```
curl -sI $BASE/health
Observed: X-Powered-By: Express. No Content-Security-Policy, X-Frame-Options,
Strict-Transport-Security, or X-Content-Type-Options. Twelve rapid failed logins produced no 429.
```

Verification

Confirmed on staging. The response identifies the framework and carries none of the expected security headers, and twelve rapid failed logins produced no throttling. Validation errors are generic and do not leak internals, and unknown routes return the framework default page.

Recommendation

Add a security-header middleware, wire rate limiting with stricter limits on authentication and upload routes, set an explicit request-body limit, and stream uploads to storage rather than buffering them.

Medium Severity Issues

The cache tier is an untimed hard dependency on the entire case lifecycle

Resolved

Confirmed on staging (dynamic test)

Path

```
recovery-backend/src/controllers/case.controller.ts:102; config/config.ts:25-30
```

Description

Every case read and every state-changing case operation awaits a cache call with no per-command timeout and no fallback to the primary database. The read path queries the cache first, and the court-order and fulfillment paths await the cache only to invalidate a key, so a non-cache operation is gated on cache availability. The client is constructed with no command timeout.

```
try {  
    const cachedCase = await config.redisClient.get(cacheKey);  
  
    if(cachedCase) {  
        res.status(200).json(JSON.parse(cachedCase));  
    } else {  
        const recovery = await JuratCaseModel.findOne({  
            where: { caseId }, logging: false  
        })  
  
        const transactions = await JuratTransactionModel.findAll({  
            where: { caseId }, logging: false  
        })  
  
        await config.redisClient.set(  
            cacheKey,  
            JSON.stringify({ recovery, transactions }),  
            {EX: RecoveryController.CACHE_TTL}  
        );  
        res.status(200).json({ recovery, transactions });  
    }  
}
```

The cache tier is an untimed hard dependency on the entire case lifecycle *(continued)*

Steps to Reproduce

With the cache unavailable, call the case read and the court-order routes and observe the gateway timeout.

Proof of Concept

```
curl -s -b jar.txt $BASE/recovery-cases/<id> # 504
curl -s -b jar.txt -X PATCH $BASE/recovery-cases/<jrcHash>/court-order -d
' {... } ' # times out
```

Verification

Observed on staging. With the cache unavailable, the case-read and court-order routes returned gateway timeouts, which is the same outage that blocked the single-case read confirmation.

Recommendation

Add per-command timeouts and fall back to the primary database on cache failure, and never gate a write path on cache availability.

Medium Severity Issues

A single hot signer key held in an environment variable authorizes all on-chain recovery

Resolved

Confirmed in source (not live-testable this phase)

Path

`recovery-backend/src/config/web3.ts:1-9`

Description

The platform signer is a plaintext private key read from an environment variable and instantiated on every fulfillment. This single key is the on-chain authority for recovery execution across all cases, with no hardware or key-management service, no per-transaction approval, and no key separation.

```
export const providers = {
  "base": "https://base.rpc.subquery.network/public",
  "baseSepolia": "https://base-sepolia-rpc.publicnode.com",
}

export const signers = {
  "base": process.env.SIGNER_BASE_PK!,
  "baseSepolia": process.env.SIGNER_BASESEPOLIA_PK!
}
```

Verification

Not observable from the outside. Confirmed in source. The deployed key-handling posture should be verified on the host.

Recommendation

Move signing to a key-management service or a multi-party signer so raw key material is never exposed, separate keys per chain and per privilege, and add rotation and alerting.

Medium Severity Issues

Court-order documents are pinned to public IPFS

Confirmed in source (not live-testable this phase)

Resolved

Path

```
recovery-backend/src/services/ipfs.services.ts:33-39
```

Description

Court-order documents are pinned to public IPFS. Anyone who learns or enumerates the content identifier can retrieve sensitive probate material with no authentication.

```
const upload = await pinata.upload.public.file(  
  new File(  
    [blob],  
    `${jrcHash}.pdf`,  
    {type: "pdf"}  
  )  
);
```

Verification

The content identifier is produced only after administrative fulfillment, which could not be executed, so this was confirmed in source. Retrieval should be checked live once fulfillment is available.

Recommendation

Use private or encrypted pinning for court documents, minimize their contents, and treat content identifiers as sensitive.

Medium Severity Issues

Server-side request forgery and untrusted document parsing in the court-records scraper

Resolved

Confirmed in source; live proof requires privileged credentials

Path

`recovery-backend/src/services/court.services.ts:350-572`

Description

The court-records scraper extracts URLs from returned HTML and re-requests them within an authenticated session, with no host allow-list, and it parses the fetched bytes as a document and writes them to disk. Because the followed content is attacker-influenceable, a crafted docket can redirect the authenticated fetches and feed arbitrary bytes to the parser.

```
const aHref = $("a")
  .filter( (_, el) => {
    const text = $(el).text();
    return !!text.match(/.*Docket (Report|Sheet).*/i);
  })
  .attr("href");

if (aHref) {
  const { data } = await client.get<string>(new URL(aHref, iqMenuU
rl).href);
```

Verification

The scraper runs only during administrative fulfillment, which returned forbidden, so this was confirmed in source. The session cookie is scoped to the court domain, which limits credential leakage but not the request forgery itself.

Recommendation

Constrain every followed URL to the court domain before requesting, disable cross-host redirects, enforce size limits, and sandbox the document parser.

Low Severity Issues

Signed recovery and court-order messages carry no nonce, deadline, or case binding

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/services/ethers.services.ts:129-152`

Description

None of the signed messages include a nonce, an expiry, or a unique case identifier. Exact re-submission of a creation message is prevented only by a uniqueness constraint on the derived hash, and the court-information signature contains no case binding at all, so it can be reused across cases that share the same lost account and chain.

```
verifySignedCourtInfoMessage(  
  signer: string,  
  info: JuratCaseCourtInput,  
  chainId: number,  
  lostAccount: AddressLike,  
  signature: SignatureLike  
): boolean {  
  const domain: SignatureDomain = {  
    name: "Jurat Signature Domain Name",  
    version: "1.0.0",  
    chainId: chainId,  
    verifyingContract: lostAccount.toString()  
  };  
  
  const msg = {  
    title: info.title,  
    fullCaseNumber: info.fullCaseNumber,  
    courtId: info.courtId,  
    docketNumber: info.docketNumber  
  }  
  
  const expSigner = ethers.verifyTypedData(domain, caseCourtInformationSignatureType, msg, signature)  
  return ethers.getAddress(signer) == expSigner;  
}
```

Signed recovery and court-order messages carry no nonce, deadline, or case binding *(continued)*

Steps to Reproduce

Submit a valid signed creation message, then submit the identical bytes again.

Proof of Concept

```
First submission: 200 {"caseId":"0f6ee6c7..."}.
```

Identical resubmission: 500 INTERNAL_SERVER_ERROR, stopped only by the uniqueness constraint on the derived hash rather than by any nonce.

Verification

Confirmed on staging. An identical signed intent submitted twice re-passed signature verification and was stopped only by the database constraint, surfacing as an ungraceful server error rather than a clean conflict.

Recommendation

Add a server-issued single-use nonce, a deadline, the chain identifier, and the specific case identifier to every signed message, and verify and consume the nonce server-side.

Low Severity Issues

Terms and will signatures are verified over a stringified payload and are replayable

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/services/ethers.services.ts:177-197`

Description

The terms and will signatures are verified with a personal-sign scheme over a stringified payload, with no domain separation, no nonce, and no owner or time binding. The terms payload is a constant, so the same signature is produced every time, and the will signature encodes the entire file as a numeric array.

```
verifySignerAgreement(  
  signer: string,  
  signature: SignatureLike  
) {  
  const msg = `${JSON.stringify(termSections)}`  
  const expSigner = ethers.verifyMessage(msg, signature);  
  
  return ethers.getAddress(signer) == expSigner;  
}  
  
verifySignerWillUpload(  
  owner: string,  
  willFileBuffer: Buffer,  
  signature: SignatureLike  
) {  
  const msg = `${JSON.stringify(willFileBuffer)}`  
  const expSigner = ethers.verifyMessage(msg, signature);  
  
  console.log(expSigner, owner);  
  return ethers.getAddress(owner) == expSigner;  
}
```

Terms and will signatures are verified over a stringified payload and are replayable *(continued)*

Steps to Reproduce

1. Sign the terms message and submit it for an owner. 2. Resubmit the identical signature. 3. As an unrelated user, submit the same owner and signature pair.

Proof of Concept

```
curl -s -b jar.txt -X POST $BASE/estate-profiles/0x<owner>/agreement \
  -d '{"email":"a@b.c","signature":"0x<personal_sign over the terms blob>"}'
```

Observed: 200 on first submission, 200 again on replay, and 200 when a different account replays the same owner and signature pair. The signature is static and cross-principal replayable.

Verification

Confirmed on staging. A terms signature was accepted, then replayed by the same user and by an unrelated user against the same owner, in each case successfully, which demonstrates a static and cross-principal-replayable credential.

Recommendation

Sign typed data over a hash commitment that includes the owner, the chain, a content hash, a nonce, and a deadline, and never ask the wallet to sign an entire file body.

Low Severity Issues

Missing owner authorization on the webhook delivery-history endpoint

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/controllers/webhook.controller.ts:73-90`

Description

The webhook delivery-history endpoint filters only by the identifier in the path and never restricts to the owner, unlike the sibling list and delete endpoints.

```
async getWebhookDeliveryHistory(req: Request, res: Response, next: NextFunction) {
    const auth = res.locals.auth as AuthPayload;

    const inputs = WebhookDeliveryHistorySchema.safeParse({ id: req.params.id });
    if(!inputs.success) throw new InputValidationError();

    const { id } = inputs.data;

    try {
        const webhookDeliveries = await JuratWebhookDeliveryModel.findAndCountAll(
            { where: { webhookId: id }, logging: false },
        )

        res.status(200).json({ rows: webhookDeliveries.rows })
    } catch(err) {
        next(err)
    }
}
```


Missing owner authorization on the webhook delivery-history endpoint

(continued)

Steps to Reproduce

1. As user A, register a webhook and note its identifier. 2. As an unrelated user B, request that webhook's delivery history.

Proof of Concept

```
curl -s -b jarB.txt $BASE/webhooks/<A-webhook-id>/history  
Observed: 200 {"rows":[]} for the non-owner, and 200 for a non-existent identifier, so there is no ownership or existence gate. Unauthenticated requests return 401.
```

Verification

Confirmed on staging. A second tenant read the first tenant's webhook history successfully, and a non-existent identifier also returned success, which shows there is no ownership or existence gate. The impact is bounded today because histories were empty and payloads are minimal.

Recommendation

Join the delivery query through the owning webhook and require the owner identity before returning history.

Low Severity Issues

Privacy-code signature domain omits the chain identifier and fixes the verifying contract to the zero address

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/services/ethers.services.ts:154-174`

Description

The privacy-code signature domain omits the chain identifier and fixes the verifying contract to the zero address, so the domain separator is identical across chains and deployments. With no nonce, a captured signature is replayable indefinitely.

```
verifySignedPrivacyCodeCreationMessage(  
  signer: string,  
  info: PrivacyCodeInput  
): boolean {  
  const domain: Partial<SignatureDomain> = {  
    name: "Jurat Signature Domain Name",  
    version: "1.0.0",  
    //chainId: 0,  
    verifyingContract: ethers.ZeroAddress  
  };  
  
  const msg = {  
    signer,  
    juratWalletAddress: info.juratWalletAddress,  
    name: info.name,  
    state: info.state  
  }  
  
  const expSigner = ethers.verifyTypedData(domain, privacyCodeSignatureType, msg, info.signature)  
  return ethers.getAddress(signer) == expSigner;  
}
```

Privacy-code signature domain omits the chain identifier and fixes the verifying contract to the zero address *(continued)*

Steps to Reproduce

Sign the privacy-code message with the domain the backend uses (no chain identifier, zero-address verifying contract), submit it, replay it, then submit the same message with a chain identifier added to the domain.

Proof of Concept

First submission: 201. Replay: 201, producing a duplicate record. The same message signed with a chain identifier in the domain returns 401 INVALID_SIGNER, which proves the deployed verifier omits it.

Verification

Confirmed on staging. The signature was accepted, then replayed to create a duplicate record, and the same message signed with a chain identifier added to the domain was rejected, which proves the deployed verifier omits it. The message does bind the signer to the recovered address, which caps the severity.

Recommendation

Include the chain identifier and a real verifying contract in the domain and add a per-owner nonce to the signed message.

Low Severity Issues

Unvalidated transaction value produces an uncaught error before signature verification

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/services/jrc.services.ts:8-48`

Description

The transaction value is validated only as a string, so a non-numeric or decimal value passes schema validation and then reaches a conversion that throws. That conversion runs before the signature check and outside the handler's error handling.

```
export const TransactionInputSchema = zod.object({
  dest: zod.string().regex(/^0x[a-fA-F0-9]{40}$/, {message: hexAddressError}),
  value: zod.string(),
  data: zod.string().regex(/^0x[a-fA-F0-9]*$/, {message: hexStringError})
})
```

Unvalidated transaction value produces an uncaught error before signature verification *(continued)*

Steps to Reproduce

Submit a creation request whose transaction value is a non-numeric or decimal string. The conversion runs before the signature check, so any signature value can be used.

Proof of Concept

```
curl -s -b jar.txt -X POST $BASE/recovery-cases/new-case -H 'Content-Type: application/json' \
  -d '{"case":{...},"transactions":[{"dest":"0x00..dEaD","value":"abc","data":"0x"}],"signature":"0x.."}'
Observed: value "abc" and "1e9" return 500 (generic, no stack); a numeric value returns 401 INVALID_SIGNER;
a baseline value of "0" with a valid signature returns 200.
```

Verification

Confirmed on staging. A non-numeric and a decimal value each produced a generic server error with no stack trace, while a numeric value returned a clean signer error and a baseline value succeeded. The error is generic and nothing is persisted.

Recommendation

Apply the unsigned-integer schema to the value on the creation path so it returns a client error rather than a server error, and validate it before the signature step.

Low Severity Issues

The change-password route has no authentication middleware and no session binding

Resolved

Confirmed on staging (dynamic test)

Path

```
recovery-backend/src/routes/auth.ts:31; controllers/auth.controller.ts:337-362
```

Description

The change-password route is registered with no authentication middleware, and its handler never reads the authenticated identity. It looks up the target account by the email in the request body, verifies the current password, and overwrites it. There is no binding to the caller's session and no session is required at all. This also corrects an earlier characterization of this route as cookie-authenticated.

```
this.router.patch('/forgot-password', authController.forgotPassword);
      this.router.patch('/recover-password', authController.recoverPassword);
      this.router.patch('/change-password', authController.changePassword);
    }
```

The change-password route has no authentication middleware and no session binding *(continued)*

Steps to Reproduce

Send a change-password request with no cookie and no authorization header, providing the account email and its current password. Then log in with the new password.

Proof of Concept

```
curl -s -X PATCH $BASE/auth/change-password -H 'Content-Type: application/json' \
  -d '{"email":"tester@example.com","password":"Passw0rd!x",
      "newPassword":"Passw0rd!y","confirmPassword":"Passw0rd!y"}'
```

Observed: 200 {"message":"User password updated"} with no authentication. Login with the new password then returns 200.

Verification

Confirmed on staging. A request with no cookie and no authorization header changed a password given the account email and current password, and a subsequent login with the new password succeeded.

Recommendation

Place the route behind the authentication middleware and change the password of the authenticated account rather than an account named in the body.

Low Severity Issues

Password change and reset do not revoke existing API keys or sessions

Resolved

Confirmed on staging (dynamic test)

Path

```
recovery-backend/src/controllers/auth.controller.ts:322-356; services/apiKey.services.ts:60
```

Description

Neither the password change nor the reset invalidates other credentials. API keys have no expiry and there is no session revocation, and a routine that revokes all of a user's API keys exists in the codebase but is never called from these flows.

Steps to Reproduce

1. Issue an API key over a cookie session. 2. Change the account password. 3. Use the API key with no cookie.

Proof of Concept

```
curl -s -H 'x-api-key: <key>' -H 'x-api-secret: <secret>' $BASE/auth/api-key  
s  
Observed: 200 after the password change. The key remains valid, so the reset  
does not revoke it.
```

Verification

Confirmed on staging. An API key issued before a password change remained fully valid afterward, which defeats the primary purpose of a password reset during account recovery.

Recommendation

Revoke API keys and invalidate active sessions on password change and reset.

Low Severity Issues

An estate-profile record is created for an arbitrary owner before the signature check

Resolved

Confirmed on staging (dynamic test)

Path

`recovery-backend/src/controllers/estate.controller.ts:126-133`

Description

The estate endpoints create the profile record before verifying the signature, with no surrounding transaction. An unauthorized caller can therefore create a persistent record keyed to any owner address, even though the request itself then fails authorization.

```
const [profile] = await JuratEstateProfileModel.findOrCreate({
  where: { ownerAddress: ownerAddress },
  defaults: getEstateDefaults(ownerAddress),
  logging: false,
});

const verifiedMsg = ethersServices.verifySignerAgreement(ownerAddress, payload.signature);
if (!verifiedMsg) throw new InvalidSignerError();
```

An estate-profile record is created for an arbitrary owner before the signature check *(continued)*

Steps to Reproduce

As a non-owner, submit an agreement for an owner address you do not control. The signature check fails, then read the owner profile.

Proof of Concept

```
curl -s -b jar.txt -X POST $BASE/estate-profiles/0xb4bB..52Cd/agreement \
  -d '{"email":"a@b.c","signature":"0x<signed by a different key>"}' # 401
INVALID_SIGNER
curl -s -b jar.txt $BASE/estate-profiles/0xb4bB..52Cd
# 200 non-null profile
The profile record was created despite the failed authorization.
```

Verification

Confirmed on staging. A request signed by a non-owner returned an invalid-signer error, yet a subsequent read of that owner address returned a non-null profile, which shows the record was created despite the failed check.

Recommendation

Verify the signature before any record is created, and wrap the operation so a failed authorization leaves no persistent state.

Low Severity Issues

Credential-less admin access via hasBackend empty-string footgun (contingent)

Resolved

Confirmed in source (not live-testable this phase)

Path

```
manual-review-dashboard/src/services/apiClient.ts:1,39
```

Description

The dashboard resolves its backend URL with a default that is applied only for a missing value, not for an empty string, and the shipped configuration file sets it to empty. A build produced with that value treats the backend as absent, and the sign-in routine then resolves any credentials as authenticated and skips the role check.

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? "http://localhost:3000";
```

Steps to Reproduce

Verification requires the deployed dashboard bundle. Inspect the built JavaScript for the configured backend URL. If it is empty, the sign-in path resolves any credentials as authenticated.

Proof of Concept

```
grep -o 'VITE_API_BASE_URL[^,}]*' dist/assets/*.js
```

An empty value causes the client to treat the backend as absent and to authenticate any credentials.

Verification

Confirmed in source. No deployed dashboard was available to test, so the severity is contingent on the build configuration and the serving arrangement.

Credential-less admin access via hasBackend empty-string footgun (contingent) *(continued)*

Recommendation

Treat an empty or whitespace value as unset and fail closed, remove the no-backend path from any production build, and assert a valid backend URL at build time.

Low Severity Issues

New administrator passwords are returned in the HTTP response body

Resolved

Confirmed in source; live proof requires privileged credentials

Path

```
recovery-backend/src/controllers/auth.controller.ts:213,231
```

Description

The generated password for a newly created administrator is returned in the response body. The response lands in proxy logs, browser history, and request-capture tools. The account is also created without email verification and without a forced password change.

```
if(!userCreated) throw new UserRegistrationError();

// Note: should use the email service to send the password
res.status(201).json({ email: adminEmail, password: newPassword
})
```

Verification

The endpoint is restricted to the master role and returned forbidden to a standard user, so the echo was confirmed in source. The response line sits after the role check, so a master would receive the plaintext credential.

Recommendation

Never return the password. Deliver a one-time invitation or reset link out of band and require a password change and email verification before the administrator can act.

Low Severity Issues

The administrative dashboard is exposed to cross-site request forgery

Resolved

Confirmed in source (not live-testable this phase)

Path

```
manual-review-dashboard/src/services/apiClient.ts:9-23
```

Description

Every dashboard request includes credentials to a different origin, and the state-changing administrative actions carry no anti-forgery token. Combined with the cross-site cookie settings on the backend, an authenticated reviewer who visits a malicious page could have fulfillment or rejection requests issued under their session.

Verification

Confirmed in source. Full confirmation depends on the deployed cross-origin allow-list, which requires a running dashboard and backend pairing.

Recommendation

Prefer a same-origin API with strict same-site cookies, or require an anti-forgery token on all state-changing routes and validate the origin, and keep the allow-list minimal.

Low Severity Issues

The wallet signing application ships without a content security policy or frame protection

Resolved

Confirmed in source (not live-testable this phase)

Path

`minimal-frontend/next.config.ts:1-7`

Description

The wallet application ships with no content security policy and no frame protection. Because the application produces typed-data signatures, the absence of frame protection exposes the signing flow to interface-redress attacks.

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
};

export default nextConfig;
```

Steps to Reproduce

Verification requires the deployed frontend. Request a page and inspect the response headers for a content security policy and frame protection.

Proof of Concept

```
curl -sI https://<frontend-host>/ # expected to lack Content-Security-Policy and X-Frame-Options
```

Verification

Confirmed in source. No deployed frontend was available to test.

Recommendation

Add a content security policy that restricts framing and script sources, along with frame options and strict transport security.

Low Severity Issues

Provider RPC keys can be compiled into the public client bundle

Resolved

Confirmed in source (not live-testable this phase)

Path

```
minimal-frontend/src/utils/config.ts:6-9
```

Description

Environment values prefixed for public exposure are compiled into the client bundle. If an operator configures a keyed provider RPC URL there, the key is published to every visitor.

Steps to Reproduce

Verification requires the deployed frontend bundle. Search it for keyed provider URLs.

Proof of Concept

```
grep -Eo 'https://[a-z-]+\.(alchemy|infura|quicknode)[^"]+' dist/assets/*.js
```

Verification

Confirmed in source and latent. The shipped sample leaves these unset and falls back to public endpoints, so live confirmation requires the deployed bundle.

Recommendation

Do not place keyed provider URLs in publicly exposed values. Proxy provider calls through a server route that injects the key.

Low Severity Issues

Dashboard authentication state is a forgeable client-side flag

Resolved

Confirmed in source (not live-testable this phase)

Path

`manual-review-dashboard/src/context/AuthContext.tsx:18-44`

Description

Whether the dashboard renders is decided by a client-side flag that any user can set. The login screen is a convenience rather than a control, and every privileged action depends entirely on backend enforcement.

```
const [isAuthenticated, setIsAuthenticated] = useState(() => {
  if (typeof window === "undefined") {
    return false;
  }
  const expiry = Number(window.localStorage.getItem("jurat-auth-expiry")
?? 0);
  const isValid = window.localStorage.getItem("jurat-authenticated") ===
"true" && Date.now() < expiry;
  if (!isValid) {
    window.localStorage.removeItem("jurat-authenticated");
    window.localStorage.removeItem("jurat-auth-expiry");
  }
  return isValid;
});
```

Dashboard authentication state is a forgeable client-side flag *(continued)***Steps to Reproduce**

Verification requires the deployed dashboard. In the browser console, set the client authentication flag and reload to render the privileged interface.

Proof of Concept

```
localStorage.setItem('jurat-authenticated', 'true');  
localStorage.setItem('jurat-auth-expiry', String(Date.now()+9e9));  
location.reload(); // renders the dashboard without a login
```

Verification

Confirmed in source. No deployed dashboard was available to exercise the flag, and the backend enforcement it depends on is weakened by the case-read and fulfillment-guard findings.

Recommendation

Derive authentication state at runtime from a verified session call rather than a stored flag, and re-verify before rendering privileged interface elements.

Low Severity Issues

The reviewer role is enforced only in the client and logout does not end the server session

Resolved

Confirmed in source (not live-testable this phase)

Path

```
manual-review-dashboard/src/services/authService.ts:37-42
```

Description

The administrator role is read in the client and enforced there, and the sign-out routine does not call the backend, so a rejected session cookie is never invalidated. Role enforcement is therefore advisory in the client.

Steps to Reproduce

Verification requires the deployed dashboard and backend pairing. Confirm that a non-administrator session can call a privileged route directly, and that sign-out does not invalidate the server session.

Proof of Concept

```
curl -s -b nonadmin.txt -X PATCH $BASE/recovery-cases/review/fulfill/<id>  
The client role check is advisory; the server must enforce the role.
```

Verification

Confirmed in source. Depends on the backend enforcing the role on every privileged route, which requires a deployed pairing to exercise.

Recommendation

Keep the client check for usability but enforce the role on the server for every privileged route, and make sign-out invalidate the server session.

Low Severity Issues

The administrative single-page application ships without security response headers

Resolved

Confirmed in source (not live-testable this phase)

Path

```
manual-review-dashboard/vite.config.ts:1-7
```

Description

The administrative single-page application is served as a static bundle with no content security policy, frame protection, or related headers.

Verification

Confirmed in source. No deployed dashboard was available to test.

Recommendation

Serve the dashboard with a strict content security policy, frame protection, and strict transport security at the hosting layer.

Low Severity Issues

The database schema is auto-altered on every service start

Resolved

Confirmed in source (not live-testable this phase)

Path

recovery-backend/src/index.ts:29-37

Description

The service auto-alters all database tables on every start, which risks schema drift and destructive column changes on a production start.

```
await JuratUserModel.sync({ alter: true, logging: false });
await JuratCaseModel.sync({ alter: true, logging: false });
await JuratTransactionModel.sync({ alter: true, logging: false });
await EstateProfileModel.sync({ alter: true, logging: false });
await JuratPrivacyCodeModel.sync({ alter: true, logging: false });
await privacyCodeServices.migrateLegacyPrivacyCodes();
await JuratApiKeyModel.sync({ alter: true, logging: false });
await JuratWebhookModel.sync({ alter: true, logging: false });
await JuratWebhookDeliveryModel.sync({ alter: true, logging: false });
```

Verification

Not observable from the outside. Confirmed in source.

Recommendation

Use versioned migrations and never auto-alter on a production start.

Low Severity Issues

The dashboard has no server-side logout and diverges from the backend session

Resolved

Confirmed in source (not live-testable this phase)

Path

```
manual-review-dashboard/src/services/authService.ts:67-69
```

Description

The dashboard sign-out clears only a local flag and the local expiry is independent of the backend session lifetime, so the interface can show an authenticated state after the session has expired, or keep the session alive after a local sign-out.

Verification

Confirmed in source. No deployed dashboard was available to test.

Recommendation

Implement a backend logout that invalidates the session and derive the interface state from the server rather than a local timer.

Low Severity Issues

The administrative login screen advertises a credential-less access path

Resolved

Confirmed in source (not live-testable this phase)

Path

```
manual-review-dashboard/src/components/Login.tsx:108
```

Description

The administrative login screen displays text that advertises the credential-less access path, which both documents the bypass to any visitor and is inappropriate for a privileged interface.

Verification

Confirmed in source. No deployed dashboard was available to test.

Recommendation

Remove the text and the underlying no-backend path before any non-local build.

Low Severity Issues

The will-upload signature step stringifies the file buffer before authorization

Resolved

Confirmed in source (not live-testable this phase)

Path

```
recovery-backend/src/services/ethers.services.ts:187-197
```

Description

The will-upload step verifies a signature computed over the stringified file buffer, which expands the payload before the signature and file-type checks. Combined with the in-memory upload limit and the absence of rate limiting, this is a resource-amplification vector.

Verification

Reasoned from source. No load was generated against the endpoint, in keeping with the non-destructive scope.

Recommendation

Sign a content hash rather than the file body, and enforce type and size checks before the expensive step.

Informational Severity Issues

Recovered signer and owner addresses are written to debug logs

Resolved

Confirmed in source (not live-testable this phase)

Path

```
recovery-backend/src/services/ethers.services.ts:195
```

Description

The recovered signer and owner addresses are written to debug logs during signature verification, and additional debug logging is present in the estate controller.

Verification

Not observable from the outside, since server logs were not accessible. Confirmed in source.

Recommendation

Remove the debug logging and route any necessary logging through a leveled logger with redaction.

Recommendations to Complete Verification

Several confirmed-in-source findings could not be exercised end to end in this phase. To convert the code-confirmed recovery chain into a demonstrated result and to close the remaining verification gaps, the following are required from the client.

1. **Administrative credentials.** A staging administrative or master account is the single most important item. It would allow the court-order fail-open, the forged-docket acceptance, the arbitrary-calldata execution, and the administrative guard defect to be exercised end to end.
2. **The live environment value.** Confirm the exact runtime value that selects the court-order check, and whether it is validated at startup. This settles whether the check is fail-open in production.
3. **Restored cache.** Restore the staging Redis instance to unblock the case-read authorization test and to re-run the case lifecycle without gateway timeouts.
4. **Deployed frontends.** Provide the deployed frontend and dashboard so the client-side findings can be exercised against production bundles.
5. **Contract behavior.** Provide the recovery contract or authorization to review it, so the realized magnitude of the recovery chain can be established.

Appendix A: Signing Helper for the Proofs of Concept

Several endpoints verify an EIP-712 typed-data signature. The proofs of concept for those findings produce the signature with the helper below before sending the request. The helper uses the same domain and types the backend verifies, and it demonstrates the core defect in the recovery-request finding: the signature is produced by a freshly generated key while the lost account is an address the signer does not control. The creation and court messages are EIP-712 typed data, so the signature is produced with a small helper before the request is sent. The helper uses the same domain and types the backend verifies.

```
// sign.mjs, run with ethers v6
import { ethers } from "ethers";
const wallet = ethers.Wallet.createRandom(); // the recovery (destination) key
const lost = "0xFA779e5670Fd0A2180C62ca19d2682DB9DBf5Ff4"; // an account we do not control
const domain = { name:"Jurat Signature Domain Name", version:"1.0.0", chainId:84532, verifyingContract:lost };
const types = { tx:[{name:"dest",type:"address"},{name:"value",type:"uint256"},{name:"data",type:"bytes"}],
  caseData:[{name:"lostAccount",type:"address"},{name:"recoveryAccount",type:"address"},{name:"chainId",type:"uint256"},
    {name:"reason",type:"string"},{name:"email",type:"string"},{name:"txs",type:"tx[]"}] };
const message = { lostAccount:lost, recoveryAccount:wallet.address, chainId:84532, reason:"lost key",
  email:"", txs:[{dest:"0x0000000000000000000000000000000000000000000000000000000000000000dEaD", value:0n, data:"0x"}] };
console.log(wallet.address, await wallet.signTypedData(domain, types, message));
```

Closing Summary

In this report, we have considered the security assessment of the Jurat Recovery Protocol. We performed our audit according to the procedure described above. All the issues have been fixed by the Jurat team.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

July 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com